

6-DAY INSTRUCTOR-LED TECHNICAL TRAINING

Core Python with AI-Integrated Development

A fast-track, hands-on bootcamp for experienced developers: master Python's object model, data structures, and standard library — while working AI coding assistants into every stage of the development lifecycle.

- Prior dev experience required
- Python 3.14 · VS Code
- AI Coding Assistant hands-on labs

```
day_01.py

developer =
Engineer(role="builder")

assistant =
configure_ai_assistant()

for day in course.days:
    day.learn()
    day.build_with(assistant)
    day.ship()
```

6-Day Agenda at a Glance

D1

Day 1

Conceptual Foundations, AI Coding Assistant Setup & Python/String Fundamentals

D2

Day 2

AI Coding Agents, Control Flow, Iterators & Core Data Structures (Lists, Tuples, Sets)

D3

Day 3

Dictionaries, Nested & Extended Data Structures, and File Handling

D4

Day 4

Modules & Packages, OOP, Comprehensions, and Exceptions/Debugging/AI-Assisted Testing

D5

Day 5

Core Libraries, Automation, and AI Tools for Developers (Copilot, Context Engineering & Adoption)

D6

Day 6

Capstone Project + Evaluation

DAY 3

Dictionaries, Nested & Extended Data Structures, and File Handling

- Working with Dictionaries
- Extended Data Structures
- Nested Data Structures (continuation of Working with Dictionaries)
- File Handling

What You'll Be Able To Do

1

Create, update, and safely query dictionaries using operators and built-in methods (`.get`, `.update`, `.pop`, `in`).

2

Model hierarchical, real-world data (JSON-like API responses, configs) using nested lists, tuples, and dictionaries.

3

Select the right specialized container (array, `heapq`, `namedtuple`, `Counter`, `deque`, `defaultdict`) for a given performance or algorithmic need.

4

Read from and write to text and binary files using the correct mode, method, and iteration style.

5

Apply context managers and defensive checks (`os.path.exists`, `try/except`) to handle files safely and avoid resource leaks.

6

Combine dictionaries, nested structures, and file I/O to build a small end-to-end data-processing script.

Working with Dictionaries

A dictionary (dict) is Python's built-in hash map — an unordered (insertion-ordered since 3.7) collection of unique key-value pairs.

THE PROBLEM IT SOLVES

Lists and tuples only give positional access; dictionaries let you look up values by meaningful keys in $O(1)$ average time.

WHY IT MATTERS

Dictionaries back most real Python data modeling — JSON payloads, configs, function kwargs, and caches all rely on them.

WHERE IT'S USED

REST API responses, application settings, database records, caching layers, and counting/aggregation logic.

REAL-WORLD ANALOGY

"Like a phone book — you look up a person's number by their name (key), not by their position in the book."

Dictionary Fundamentals

KEY TERMINOLOGY

- key — unique, hashable identifier (str, int, tuple, etc.)
- value — any object, mutable or immutable
- key-value pair — one entry in the dict
- hashable — must implement `__hash__`; lists/dicts cannot be keys

CREATING & UPDATING

- Literal syntax: `{}` or `{key: value, ...}`
- `dict()` constructor from kwargs or an iterable of pairs
- Assign/update a key: `d[key] = value` (adds or overwrites)
- Dicts are mutable — updating in place doesn't create a new object

IMPORTANT RULES

- Keys must be unique — a repeated key overwrites the earlier value
- Keys must be hashable/immutable; values can be anything
- Insertion order is preserved and guaranteed since Python 3.7

DEVELOPER PERSPECTIVE

- Think of a dict as an in-memory hash map, same complexity guarantees
- Prefer `.get()` over `d[key]` when a missing key is expected, not exceptional

Creating, Accessing & Updating

```
syntax.py

# Creating
person = {"name": "Alice", "age": 30}
empty = {}
from_pairs = dict([("a", 1), ("b", 2)])

# Assigning / updating
person["age"] = 31          # update existing key
person["city"] = "Berlin"   # add new key

# Accessing
print(person["name"])       # KeyError if missing
print(person.get("email"))  # None if missing, no error
```

HOW IT READS

- {} creates an empty dict; {k: v} creates one with entries
- dict() can build from an iterable of (key, value) pairs
- d[key] = value both inserts and overwrites — same syntax
- d[key] raises KeyError on a missing key; .get(key) returns None instead

Basic Example

```
example.py

inventory = {"apples": 50, "bananas": 30}

inventory["oranges"] = 20          # add new key
inventory["apples"] = 45          # update existing key

for fruit, count in inventory.items():
    print(f"{fruit}: {count}")
```

WHAT'S HAPPENING

- Dict literal starts with two entries
- New key 'oranges' is added via assignment
- Existing key 'apples' is overwritten, not duplicated
- `.items()` yields (key, value) pairs for iteration

CONSOLE OUTPUT

```
apples: 45
bananas: 30
oranges: 20
```

Practical Example

```
practical_example.py

config = {"timeout": 30, "retries": 3, "debug": False}

# .update() merges another dict / kwargs in place
config.update({"timeout": 60, "verbose": True})

# .keys(), .values()
print(list(config.keys()))
print(list(config.values()))

# .get() with a default avoids KeyError
max_conn = config.get("max_connections", 100)

# 'in' checks key membership, not values
if "debug" in config:
    print("debug flag present:", config["debug"])
```

WHAT'S HAPPENING

- `.update()` merges keys, overwriting shared ones ('timeout')
- `.keys()/.values()` return dynamic views, not lists (wrap in `list()` to print)
- `.get(key, default)` avoids a `KeyError` for optional settings
- `'in'` tests key presence in $O(1)$; combine with `.pop()` to remove-and-use a value

CONSOLE OUTPUT

```
['timeout', 'retries', 'debug', 'verbose']
[60, 3, False, True]
debug flag present: False
removed: 3 -> config now: {'timeout': 60,
'debug': False, 'verbose': True}
```

Nested Data Structures — A Preview

- Dict values aren't limited to scalars — they can be lists, tuples, or other dicts
- Example: `{"user": {"id": 1, "roles": ["admin", "editor"]}}`
- This lets a single dict model an entire JSON document
- Full nesting patterns (list-of-dicts, dict-of-dicts, etc.) are covered next

Common Mistakes

✗ INCORRECT

```
value = d["missing_key"]
```

✓ CORRECT

```
value = d.get("missing_key", default)
```

✗ INCORRECT

```
for k in d.keys():  
    print(d[k])
```

✓ CORRECT

```
for k, v in d.items():  
    print(v)
```

✗ INCORRECT

```
if d.values() == "x":  
    ...
```

✓ CORRECT

```
if "x" in d:  
    ...
```

✗ INCORRECT

```
d["count"] = d["count"] + 1
```

✓ CORRECT

```
d["count"] = d.get("count", 0) + 1
```

Common Mistakes — Why It Matters

- Mistake 1: Direct indexing raises `KeyError`; `.get()` lets you supply a fallback.
- Mistake 2: `.items()` avoids a second lookup per key and reads cleaner.
- Mistake 3: `'in'` on a dict checks keys, not values — test membership on keys directly.
- Mistake 4: Guards against `KeyError` the first time the key doesn't exist yet.

Best Practices

PEP 8 & READABILITY

- Use descriptive key names, not single letters, for long-lived dicts
- Keep dict literals on multiple lines when they hold many keys

MAINTAINABILITY

- Prefer `.get()/.setdefault()` over defensive `if-key-in` checks
- Use `dataclasses/NamedTuples` instead of dicts for fixed, typed records

PERFORMANCE

- Membership tests (`'in'`) and lookups are $O(1)$ average — prefer dicts over lists for lookups

AI-Assisted Development

PROMPT

Review this dictionary-handling code for missing-key bugs and suggest safer, more Pythonic alternatives.

```
before_ai.py

def get_price(catalog, sku):
    return catalog[sku]["price"] *
catalog[sku]["qty"]
```

AI ASSISTANT SUGGESTS

- Flags that `catalog[sku]` raises `KeyError` for an unknown SKU
- Suggests `catalog.get(sku)` with an explicit `None` check before indexing further
- Recommends caching `catalog[sku]` in a local variable to avoid a double lookup
- Points out the function returns wrong results silently if 'price'/'qty' keys are missing

DEVELOPER MUST STILL VERIFY

- Confirm the suggested default behavior matches business rules (skip vs raise vs zero)
- Check performance impact of extra existence checks in hot loops
- Re-run existing unit tests after the refactor

Hands-on Exercise

PROBLEM STATEMENT

Given a dict of student names to test scores, add a new student, update one existing score, safely look up a student that may not exist, and print all students who scored above 80.

INPUT

```
scores = {"Ana": 78, "Bo": 85, "Cy": 92}
```

EXPECTED OUTPUT

Printed lines showing students above 80 (Bo, Cy) after your updates are applied.

HINTS (no full solution here — give it a try first!)

- Use `.items()` to iterate key-value pairs for the filter
- Use `.get()` with a default for the safe lookup
- Adding a key and updating a key use the same `d[key] = value` syntax

Nested Data Structures (continuation of Working with Dictionaries)

Nested data structures are containers built from other containers — tuples of tuples, lists of dicts, dicts of dicts — used to model hierarchical, real-world data.

THE PROBLEM IT SOLVES

Flat lists/dicts can't represent structured records like an API response with nested objects and arrays.

WHY IT MATTERS

Almost every JSON payload, config file, and database result set your code touches is a nested structure.

WHERE IT'S USED

REST API JSON responses, YAML/JSON config files, grouped CSV/Excel data, GraphQL results.

REAL-WORLD ANALOGY

"Like a filing cabinet with folders inside folders inside drawers — each level groups related data."

Nesting Patterns

COMMON NESTING PATTERNS

- tuple of tuples — fixed-size, immutable rows: `((1,2),(3,4))`
- tuple of list — an immutable outer shell around mutable rows
- list of tuples — mutable collection of fixed-size records
- list of lists / list-in-dict — mutable grid or record collection

HOW IT WORKS

- Access chains subscripts left to right: `data["users"][0]["roles"][1]`
- Any container can nest any other container, to any depth
- Mutability of the outer container doesn't change inner containers' mutability

IMPORTANT RULES

- Deep copying may be needed — `assignment/.copy()` only copies the outer layer
- A tuple can hold mutable elements (e.g. a list) — the tuple itself just can't be reassigned

DEVELOPER PERSPECTIVE

- Nested dicts/lists are Python's natural mapping for parsed JSON — no extra library needed
- Deeply nested access is fragile; validate structure or use `.get()` chains defensively

Nesting Syntax

```
syntax.py

# tuple of tuples
matrix_t = ((1, 2), (3, 4))

# tuple of list
mixed = ([1, 2], [3, 4])

# list of tuples
points = [(0, 0), (1, 5), (2, 8)]

# list of lists
grid = [[1, 2], [3, 4]]

# list within a dictionary
user = {"name": "Ana", "roles": ["admin", "editor"]}

# nested dictionary
account = {"user": {"id": 1, "profile": {"city": "Berlin"}}}
print(account["user"]["profile"]["city"])
```

HOW IT READS

- Outer () makes the top container immutable; inner containers keep their own type
- Chained [] indexing walks each nesting level in order
- A dict value can itself be a list, dict, tuple, or any mix of these
- `account["user"]["profile"]["city"]` resolves three nesting levels

Basic Example

example.py

```
tuple_of_tuples = ((1, 2), (3, 4))
tuple_of_list = ([1, 2], [3, 4])
list_of_tuples = [(1, 2), (3, 4)]
list_of_lists = [[1, 2], [3, 4]]

print(tuple_of_tuples[1][0])    # 3
print(tuple_of_list[0])         # [1, 2]
tuple_of_list[0].append(99)     # inner list IS mutable
print(tuple_of_list)

list_of_lists[0][1] = 20
print(list_of_lists)
```

WHAT'S HAPPENING

- Indexing [row][col] works identically across all four variants
- tuple_of_list is itself immutable, but its inner lists can still be mutated
- list_of_lists allows both outer and inner mutation

CONSOLE OUTPUT

```
3
[1, 2]
([1, 2, 99], [3, 4])
[[1, 20], [3, 4]]
```

Practical Example

```
practical_example.py

employees = {
    "E101": {
        "name": "Diego",
        "skills": ["Python", "SQL"],
        "address": {"city": "Madrid", "zip": "28001"}
    },
    "E102": {
        "name": "Priya",
        "skills": ["Python", "AWS", "Docker"],
        "address": {"city": "Pune", "zip": "411001"}
    }
}

for emp_id, info in employees.items():
    city = info["address"]["city"]
    skill_list = ", ".join(info["skills"])
```

WHAT'S HAPPENING

- Each value in 'employees' is itself a dict — a nested dictionary
- The 'skills' key holds a list nested inside a dict inside a dict
- `info["address"]["city"]` chains two nesting levels for a single value
- Appending to a nested list mutates it in place, no reassignment needed

CONSOLE OUTPUT

```
E101: Diego (Madrid) - skills: Python, SQL
E102: Priya (Pune) - skills: Python, AWS,
Docker
['Python', 'SQL', 'Docker']
```

Advanced / Developer Example

advanced_example.py

```
api_response = {
    "status": "ok",
    "data": {
        "orders": [
            {"id": 1001, "items": [{"sku": "A1", "qty": 2}, {"sku": "B2", "qty": 1}], "total": 45.50},
            {"id": 1002, "items": [{"sku": "C3", "qty": 5}], "total": 120.00}
        ]
    }
}

def total_items(order):
    return sum(item["qty"] for item in order["items"])

for order in api_response["data"]["orders"]:
    print(f"Order {order['id']}: {total_items(order)} items, ${order['total']:.2f}")
```

WHAT'S HAPPENING

- Models a realistic API JSON body: dict -> dict -> list -> dict, four levels deep
- A generator expression sums quantities from a list of dicts nested inside a dict
- This exact shape is what `json.loads()` returns for typical REST responses
- Config files (JSON/YAML) and API payloads both map naturally onto this pattern

CONSOLE OUTPUT

```
Order 1001: 3 items, $45.50
Order 1002: 5 items, $120.00
Grand total: $165.50
```

Nesting Pattern Comparison

Pattern	Example	Mutability	Typical Use
Tuple of tuples	<code>((1, 2), (3, 4))</code>	Fully immutable	Fixed coordinate pairs, constants
List of tuples	<code>[(1, 2), (3, 4)]</code>	Outer mutable, rows fixed	Growable set of fixed-size records
List of lists	<code>[[1, 2], [3, 4]]</code>	Fully mutable	Grids, matrices, editable tables
Dict of dicts / list-in-dict	<code>{"u": {"id": 1}}</code>	Fully mutable	JSON/API data, config files, records

Common Mistakes

✗ INCORRECT

```
copy2 = original_dict  
copy2["a"]["x"] = 1
```

✓ CORRECT

```
import copy  
copy2 = copy.deepcopy(original_dict)
```

✗ INCORRECT

```
data["user"]["address"]["city"]
```

✓ CORRECT

```
data.get("user", {}).get("address", {}).get("city")
```

✗ INCORRECT

```
row = (1, [2, 3])  
row[0] = 9
```

✓ CORRECT

```
row = (1, [2, 3])  
row[1].append(4)
```

Common Mistakes — Why It Matters

- Mistake 1: Plain assignment (or shallow `.copy()`) shares nested objects — mutating `copy2` mutates `original_dict` too.
- Mistake 2: Direct chained indexing raises `KeyError` the moment any level is missing.
- Mistake 3: The tuple itself can't be reassigned, but a mutable element inside it still can be.

Best Practices

PEP 8 & READABILITY

- Break deeply nested literals across multiple indented lines
- Name intermediate variables instead of chaining 4+ subscripts on one line

MAINTAINABILITY

- Validate expected shape before deep indexing (e.g. with a schema or type hints)
- Prefer dataclasses/TypedDict for structures accessed in many places

PERFORMANCE

- Avoid re-walking the same nested path repeatedly — cache the intermediate result

AI-Assisted Development

PROMPT

This function parses a nested API response. Review it for missing-key robustness and suggest a safer version.

```
before_ai.py

def get_city(response):
    return
response["data"]["user"]["address"]["city"]
```

AI ASSISTANT SUGGESTS

- Flags that any missing intermediate key crashes the whole function
- Suggests nested `.get()` calls with `{}` defaults at each level
- Recommends a helper function for repeated deep-path lookups
- Notes this pattern is common when consuming third-party/unstable APIs

DEVELOPER MUST STILL VERIFY

- Confirm what the caller should receive when data is missing (None vs default vs raise)
- Check the suggestion doesn't silently mask a real upstream bug
- Add a test case simulating a truncated API response

Hands-on Exercise

PROBLEM STATEMENT

Given a nested dict representing a company with departments, each holding a list of employee dicts, print each department's name followed by its employees' names, then compute the total employee count across all departments.

INPUT

```
company = {"Engineering": {"employees": [{"name": "Ana"}, {"name": "Bo"}]}, "Sales": {"employees": [{"name": "Cy"}]}}
```

EXPECTED OUTPUT

```
Department names with nested employee names printed, plus a  
total count of 3.
```

HINTS (no full solution here — give it a try first!)

- Outer dict keys are department names; values are dicts holding a list
- Use a nested for loop: department dict -> employees list -> employee dict
- Accumulate a running total across departments with a counter variable

Extended Data Structures

Beyond list/tuple/dict, Python's standard library ships specialized containers — array, heapq, and collections — tuned for memory efficiency, ordering, and counting.

THE PROBLEM IT SOLVES

Generic lists and dicts aren't always the most memory- or algorithm-efficient choice for numeric data, priority ordering, or counting/grouping.

WHY IT MATTERS

Choosing the right built-in structure avoids reinventing common data-structure algorithms and improves performance.

WHERE IT'S USED

Numeric buffers in data pipelines, task schedulers and pathfinding (heapq), log analytics and word counts (collections).

REAL-WORLD ANALOGY

"Like choosing the right tool from a toolbox — a wrench (list) works everywhere, but a torque wrench (array/heapq/Counter) fits its job better."

The Extended Toolkit

ARRAY VS LIST

- `array.array` stores a single, fixed C-type — much more memory-compact than list
- list can hold mixed types but pays per-object overhead for each element
- Use array for large homogeneous numeric buffers; list for general-purpose data

HEAPQ (HEAPIFY)

- Turns a list into a binary min-heap in place via `heapq.heapify()`
- `heappush`/`heappop` maintain heap order in $O(\log n)$
- Implements priority queues — smallest element always at index 0

COLLECTIONS MODULE

- `namedtuple` — tuple with named fields, still immutable
- `Counter` — dict subclass specialized for counting hashable items
- `deque` — double-ended queue, $O(1)$ appends/pops from both ends
- `defaultdict` / `OrderedDict` — auto-default values / explicit order guarantees

DEVELOPER PERSPECTIVE

- Reach for these before hand-rolling the equivalent logic yourself
- Pick based on the operation you do most: count -> `Counter`, queue -> `deque`/`heapq`

array vs list

Aspect	list	array.array
Element types	Any mixed Python objects	Single fixed C type (e.g. 'i', 'd')
Memory usage	Higher — per-object overhead	Lower — packed C-style buffer
Typical use	General-purpose collections	Large homogeneous numeric buffers
API richness	Very rich (sort, comprehensions, etc.)	Narrower, list-like subset

The array Module

```
syntax.py

from array import array

# typecode 'i' = signed int, 'd' = double
nums = array("i", [10, 20, 30])
nums.append(40)
nums.insert(0, 5)

print(nums)
print(nums[2])
print(sum(nums))
```

HOW IT READS

- First argument is a typecode constraining every element to one C type
- `append()`/`insert()` work like list methods but reject the wrong type
- array objects are iterable and support common list-like operations

Basic Example

```
example.py

from array import array

temperatures = array("d", [21.5, 22.0, 19.8])
temperatures.append(23.1)

for t in temperatures:
    print(f"{t:.1f}C")

print("average:", sum(temperatures) / len(temperatures))
```

WHAT'S HAPPENING

- 'd' typecode restricts all elements to floats (doubles)
- Iterating an array works exactly like iterating a list
- `sum()/len()` work directly on array objects, no conversion needed

CONSOLE OUTPUT

```
21.5C
22.0C
19.8C
23.1C
average: 21.6
```

heapq — Heapify

```
syntax.py

import heapq

tasks = [(3, "cleanup"), (1, "deploy"), (2, "review")]
heapq.heapify(tasks)           # in-place min-heap by first
                               # tuple element

heapq.heappush(tasks, (0, "hotfix"))
next_task = heapq.heappop(tasks) # always the smallest

print(next_task)
print(tasks)
```

HOW IT READS

- `heapify()` reorders a list in place into valid heap order, $O(n)$
- `heappush()/heappop()` insert/remove while preserving heap order, $O(\log n)$
- Tuples compare element-by-element, so (priority, label) sorts by priority first

Practical Example

```
practical_example.py

import heapq

task_queue = []
heapq.heappush(task_queue, (2, "send report"))
heapq.heappush(task_queue, (1, "fix production bug"))
heapq.heappush(task_queue, (3, "update docs"))
heapq.heappush(task_queue, (1, "restart service"))

while task_queue:
    priority, task = heapq.heappop(task_queue)
    print(f"[P{priority}] {task}")
```

WHAT'S HAPPENING

- Lower priority number is processed first — classic priority queue semantics
- heappush keeps the heap valid after every insertion, no manual sorting
- Equal priorities ('1') fall back to comparing the task string alphabetically
- Draining with a while loop always pops the current smallest item

CONSOLE OUTPUT

```
[P1] fix production bug
[P1] restart service
[P2] send report
[P3] update docs
```

The collections Module

```
syntax.py

from collections import namedtuple, Counter, deque, defaultdict, OrderedDict

Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p.x, p.y)

counts = Counter(["a", "b", "a", "c", "a"])
print(counts.most_common(2))

dq = deque([1, 2, 3])
dq.appendleft(0)
dq.append(4)
print(dq)

grouped = defaultdict(list)
grouped["fruits"].append("apple")    # no KeyError on first use

ordered = OrderedDict([("b", 2), ("a", 1)])
print(list(ordered.keys()))
```

HOW IT READS

- `namedtuple` gives dot-access fields while staying an immutable tuple
- `Counter(iterable)` tallies occurrences; `.most_common(n)` ranks them
- `deque` supports `O(1)` `appendleft()`/`append()` unlike list's `O(n)` `insert(0, ...)`
- `defaultdict(list)` auto-creates a default value for a missing key
- `OrderedDict` preserves insertion order explicitly (plain dict does too since 3.7)

Advanced / Developer Example

```
advanced_example.py

from collections import namedtuple, Counter, defaultdict

LogEntry = namedtuple("LogEntry", ["level", "message"])

raw_lines = [
    "ERROR disk full",
    "INFO service started",
    "ERROR disk full",
    "WARNING low memory",
    "ERROR connection refused",
]

entries = [LogEntry(*line.split(" ", 1)) for line in raw_lines]

level_counts = Counter(entry.level for entry in entries)
print(level_counts)

by_level = defaultdict(list)
```

WHAT'S HAPPENING

- `namedtuple('LogEntry', ...)` gives each parsed line named, readable fields
- `Counter` over a generator expression tallies log levels in one pass
- `defaultdict(list)` groups messages by level without checking key existence first
- This pattern scales directly to real log-file/monitoring pipelines

CONSOLE OUTPUT

```
Counter({'ERROR': 3, 'INFO': 1, 'WARNING': 1})
ERROR: ['disk full', 'disk full', 'connection refused']
INFO: ['service started']
WARNING: ['low memory']
```

Common Mistakes

✗ INCORRECT

```
arr = array("i", [1, 2, "3"])
```

✓ CORRECT

```
arr = array("i", [1, 2, 3])
```

✗ INCORRECT

```
my_list.insert(0, x)  # in a tight loop
```

✓ CORRECT

```
dq = deque(my_list)
dq.appendleft(x)
```

✗ INCORRECT

```
sorted(pairs)[0]  # after every insert
```

✓ CORRECT

```
heapq.heappush(heap, pair)
```

✗ INCORRECT

```
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
```

✓ CORRECT

```
from collections import Counter
counts = Counter(words)
```

Common Mistakes — Why It Matters

- Mistake 1: array enforces a single C type — mixing in a string raises `TypeError`.
- Mistake 2: `list.insert(0, ...)` is $O(n)$; `deque.appendleft` is $O(1)$ for frequent front inserts.
- Mistake 3: Re-sorting after every insert is $O(n \log n)$ each time; a heap keeps it $O(\log n)$.
- Mistake 4: Counter does the same tally in one call, more clearly and less error-prone.

Best Practices

PEP 8 & READABILITY

- Name namedtuple classes in CapWords, fields in snake_case
- Import only what you use from collections

MAINTAINABILITY

- Prefer namedtuple over plain tuples for records with 3+ fields
- Use defaultdict only when the default truly makes sense everywhere

PERFORMANCE

- Use array for large numeric datasets to cut memory footprint
- Use deque instead of list for queue-like workloads at both ends

AI-Assisted Development

PROMPT

This code counts word frequency manually and manages a task queue with `list.insert(0, ...)`. Suggest more idiomatic, efficient alternatives from the standard library.

```
before_ai.py

counts = {}
for word in words:
    if word not in counts:
        counts[word] = 0
    counts[word] += 1

queue = []
queue.insert(0, "next_job")
```

AI ASSISTANT SUGGESTS

- Suggests replacing the manual counting loop with `collections.Counter`
- Flags `queue.insert(0, ...)` as $O(n)$ and recommends `collections.deque`
- Notes `Counter` also provides `.most_common()` for free ranking
- Points out `deque.appendleft()/popleft()` give $O(1)$ front operations

DEVELOPER MUST STILL VERIFY

- Confirm behavior is identical for tie-breaking in `most_common()`
- Check downstream code doesn't rely on 'queue' being a plain list (e.g. slicing)
- Benchmark before/after if performance was the actual motivation

Hands-on Exercise

PROBLEM STATEMENT

Given a list of (priority, job_name) pairs, use `heapq` to process jobs in priority order, then use `collections.Counter` on job categories (derived from job names) to report how many jobs fall in each category.

INPUT

```
jobs = [(3, "build_report"), (1, "deploy_app"), (2, "review_code"), (1, "deploy_hotfix")]
```

EXPECTED OUTPUT

Jobs printed in ascending priority order, followed by a Counter-based tally grouped by the part of the name before the underscore.

HINTS (no full solution here — give it a try first!)

- `heapq.heapify()` then repeated `heappop()` drains jobs in priority order
- `job_name.split("_")[0]` extracts the category prefix
- Counter accepts any iterable of hashable values, including a generator expression

File Handling

File handling is Python's built-in mechanism (via `open()` and file objects) for reading from and writing to files on disk, in text or binary mode.

THE PROBLEM IT SOLVES

Programs need durable I/O with the filesystem — configs, logs, exports, and data files must persist beyond a single run.

WHY IT MATTERS

Nearly every real application reads config/data files or writes logs/reports; getting modes and cleanup wrong causes data loss or resource leaks.

WHERE IT'S USED

Log processing pipelines, CSV/JSON data ingestion, report generation, config loading, binary asset handling.

REAL-WORLD ANALOGY

"Like opening a physical filing folder — you must open it, do your reading/writing, and close it when done, or the drawer stays jammed."

File Handling Fundamentals

KEY TERMINOLOGY

- file object — returned by `open()`, the handle you read/write through
- text mode — data as str, decoded using an encoding
- binary mode ('b') — data as raw bytes, no decoding
- file pointer / cursor — current read/write position within the file

HOW IT WORKS

- `open(path, mode)` returns a file object bound to an OS-level file handle
- Reads/writes advance the file pointer automatically
- The file must be closed to flush buffers and release the OS handle

IMPORTANT RULES

- Always match mode to intent: 'r' won't create a file, 'w' truncates it
- Text mode expects str; binary mode expects bytes — mixing them raises `TypeError`

DEVELOPER PERSPECTIVE

- Always prefer 'with `open(...)`' over manual `open()/close()` pairs
- Treat missing files/permission errors as expected, not exceptional, in user-facing code

Text Files vs Binary Files

Aspect	Text Mode	Binary Mode
Data type	<code>str</code> (decoded)	<code>bytes</code> (raw)
Typical suffix	<code>'r'</code> , <code>'w'</code> , <code>'a'</code>	<code>'rb'</code> , <code>'wb'</code> , <code>'ab'</code>
Use case	<code>.txt</code> , <code>.csv</code> , <code>.log</code> , <code>.json</code> , <code>.py</code>	<code>.png</code> , <code>.zip</code> , <code>.pdf</code> , <code>.exe</code> , network payloads
Encoding	Applies (e.g. UTF-8)	None — bytes are exact

File Modes

```
syntax.py

f = open("data.txt", "r")    # read text, error if file
                             # doesn't exist
f = open("data.txt", "w")    # write text, TRUNCATES
                             # existing content
f = open("data.txt", "a")    # append text, creates
                             # file if missing
f = open("data.txt", "r+")   # read + write, no
                             # truncation
f = open("image.png", "rb")  # read binary
f = open("image.png", "wb")  # write binary, truncates
f = open("data.txt", "x")    # exclusive create, errors
                             # if it already exists
```

HOW IT READS

- 'r' is the default mode and requires the file to already exist
- 'w' and 'wb' silently erase existing file content — use with care
- 'a'/'ab' always write at the end, never overwriting prior content
- Add 'b' to any mode for binary/bytes instead of text/str
- 'r+' lets you read and write without truncating on open

Basic Example

example.py

```
with open("notes.txt", "w") as f:
    f.write("first line\n")
    f.write("second line\n")

with open("notes.txt", "r") as f:
    content = f.read()

print(content)
```

WHAT'S HAPPENING

- 'w' mode creates the file (or truncates it) before writing
- write() does not add newlines automatically — include \n explicitly
- read() returns the entire file content as a single string
- Both blocks use 'with', so each file is closed automatically

CONSOLE OUTPUT

```
first line
second line
```

read(), readline(), readlines(), write(), writelines()

```
syntax.py

with open("data.txt", "r") as f:
    whole = f.read()          # entire file as one string
    f.seek(0)
    one_line = f.readline()   # next single line, keeps
    trailing \n
    f.seek(0)
    all_lines = f.readlines() # list of lines, each keeps
    trailing \n

with open("out.txt", "w") as f:
    f.write("single string\n")
    f.writelines(["line a\n", "line b\n"])
```

HOW IT READS

- read() loads everything at once — fine for small files, wasteful for huge ones
- readline() returns one line per call, empty string at end of file
- readlines() returns a list of all lines — convenient but loads it all in memory
- writelines() writes an iterable of strings; it does NOT add newlines for you

Practical Example

```
practical_example.py

log_lines = [
    "2024-01-01 INFO service started\n",
    "2024-01-01 ERROR disk full\n",
    "2024-01-02 INFO request handled\n",
    "2024-01-02 ERROR connection refused\n",
]

with open("app.log", "w") as f:
    f.writelines(log_lines)

error_count = 0
with open("app.log", "r") as f:
    for line in f:                # file object as an iterator
        line = line.strip()
        if "ERROR" in line:
            error_count += 1
    print("Found:", line)
```

WHAT'S HAPPENING

- `writelines()` dumps prepared log lines straight to disk
- `'for line in f'` treats the file object itself as an iterator — no `readlines()` needed
- Each iteration yields one line at a time, memory-efficient for large logs
- `strip()` removes the trailing newline before matching/printing

CONSOLE OUTPUT

```
Found: 2024-01-01 ERROR disk full
Found: 2024-01-02 ERROR connection refused
Total errors: 2
```

File as Iterator



Advanced / Developer Example

```
advanced_example.py

import os

def load_records(path):
    if not os.path.exists(path):
        print(f"Skipping missing file: {path}")
        return []

    records = []
    try:
        with open(path, "r") as f:          # autocloses even if an error occurs
            for line_num, line in enumerate(f, start=1):
                parts = line.strip().split(",")
                if len(parts) != 3:
                    print(f"Line {line_num}: malformed, skipping")
                    continue
                name, dept, salary = parts
                records.append({"name": name, "dept": dept, "salary": float(salary)})
    except (OSError, ValueError) as exc:
        print(f"Failed to process {path}: {exc}")
```

WHAT'S HAPPENING

- `os.path.exists()` avoids a crash on a missing file before even opening it
- `'with open(...)'` guarantees the file closes even if an exception is raised inside the block
- `try/except (OSError, ValueError)` catches both I/O failures and bad `float()` conversions
- Malformed rows are skipped individually rather than aborting the whole file
- Calling with a missing path shows the graceful early-return path

CONSOLE OUTPUT

```
Line 3: malformed, skipping
[{'name': 'Ana', 'dept': 'Eng', 'salary': 7200.0}, {'name': 'Bo', 'dept': 'Sales', 'salary': 5400.0}]
Skipping missing file: missing.csv
[]
```

Common Mistakes

✗ INCORRECT

```
f = open("data.txt", "r")
data = f.read()
```

✓ CORRECT

```
with open("data.txt", "r") as f:
    data = f.read()
```

✗ INCORRECT

```
open("report.txt", "w") # to add a line to an existing report
```

✓ CORRECT

```
open("report.txt", "a")
```

✗ INCORRECT

```
with open(path) as f:
    data = f.read() # path may not exist
```

✓ CORRECT

```
if os.path.exists(path):
    with open(path) as f:
        data = f.read()
```

✗ INCORRECT

```
with open("img.png", "r") as f:
    data = f.read()
```

✓ CORRECT

```
with open("img.png", "rb") as f:
    data = f.read()
```

Common Mistakes — Why It Matters

- Mistake 1: Without 'with', a forgotten/never-reached `close()` leaks the OS file handle.
- Mistake 2: 'w' truncates the file first — you lose everything already written.
- Mistake 3: Checking (or wrapping in `try/except FileNotFoundError`) avoids an unhandled crash.
- Mistake 4: Binary content read in text mode raises a decode error or corrupts the data.

Best Practices

PEP 8 & READABILITY

- Always use 'with open(...) as f:' instead of manual close()
- Name file variables/handles descriptively in longer functions

MAINTAINABILITY

- Specify encoding explicitly: `open(path, "r", encoding="utf-8")`
- Wrap file I/O in try/except around the specific exceptions you expect

PERFORMANCE

- Iterate line-by-line ('for line in f') instead of `readlines()` for large files

SECURITY

- Validate/sanitize file paths from user input before opening them
- Avoid overwriting files blindly with 'w' — confirm intent or check existence first

AI-Assisted Development

PROMPT

Review this file-handling function for resource leaks, missing error handling, and mode mistakes.

```
before_ai.py

def read_config(path):
    f = open(path, "w")
    data = f.read()
    return data
```

AI ASSISTANT SUGGESTS

- Flags that mode 'w' truncates the file instead of reading it — should be 'r'
- Flags the missing file.close() / lack of a 'with' block, risking a handle leak
- Notes there's no handling for FileNotFoundError or PermissionError
- Suggests specifying encoding explicitly for portability
- Recommends returning None or raising a clear error on failure, not crashing unhandled

DEVELOPER MUST STILL VERIFY

- Confirm the corrected mode matches actual intent (read vs write vs append)
- Verify exception handling doesn't swallow errors the caller needs to know about
- Test with a genuinely missing file and a permission-denied path

Hands-on Exercise

PROBLEM STATEMENT

Write a function that reads a text file of comma-separated 'name,score' lines, skips any malformed lines, computes the average score, and writes a one-line summary report to a new output file — handling a missing input file gracefully.

INPUT

```
students.txt containing:  
Ana,88  
Bo,92  
corrupted_line  
Cy,79
```

EXPECTED OUTPUT

```
summary.txt containing something like: 'Processed 3 students,  
average score: 86.33' and a console message noting the  
skipped malformed line.
```

HINTS (no full solution here — give it a try first!)

- Use `os.path.exists()` or `try/except FileNotFoundError` for the missing-file case
- Iterate the input file object directly with a `for` loop
- Open the output file in 'w' mode and use `write()` with the computed summary string

Day Summary

CONCEPTS LEARNED

- Dictionaries as Python's hash-map type: keys, values, and mutability
- Nesting containers within containers to model structured/hierarchical data
- Specialized standard-library containers: array, heapq, and collections
- The file object as a handle over an OS-level resource, in text or binary mode

KEY SYNTAX

- {}, dict(), d[key]=value, .get(), .update(), .pop(), .keys()/ .values()/ .items(), 'in'
- Nested literals: ((...), (...)), [(...)], [[...]], {"k": {...}}, chained [][] access
- array("typecode", [...]); heapq.heapify/heappush/heappop; namedtuple/Counter/deque/defaultdict
- open(path, mode), read()/readline()/readlines()/write()/writelines(), 'with' statement, 'for line in f'

KEY TAKEAWAYS

- Use .get()/ .pop() with defaults instead of raw indexing when a key may be absent
- Nested structures mirror JSON almost exactly — no extra parsing library needed
- Pick a container by the operation you do most often, not out of habit
- Always open files with 'with', check existence, and catch specific exceptions

PRACTICAL SKILLS GAINED

- Parsing and building nested JSON-like data in pure Python
- Implementing a priority queue with heapq
- Counting and grouping data with Counter/defaultdict
- Reading, writing, and safely processing text files line by line

Nested Records to File Report

Your team receives a raw text export of employee records (one per line) that must be parsed into a nested in-memory structure, summarized per department, and written out as a clean report file for stakeholders.

REQUIREMENTS

- Read the input text file using 'with open(...)' (autoclose) and iterate it as a file object
- Skip and log (print) any malformed line instead of crashing
- Parse each valid line into a nested dictionary keyed by department, each holding a list of employee dicts (name, salary)
- Use dictionary methods (.get()/setdefault() or defaultdict) to build the nested structure without KeyErrors
- Use collections.Counter to count how many employees are in each department
- Compute each department's average salary
- Write a formatted summary report to a new output text file using write()/writelines()
- Handle a missing input file gracefully using os.path.exists() or try/except FileNotFoundError

CONSTRAINTS

- Do not use any third-party libraries — standard library only
- The nested structure must be a dict of departments to lists of employee dicts (or equivalent nested form)
- The program must not crash if employees.txt does not exist

Nested Records to File Report — Input / Output

SAMPLE INPUT

```
employees.txt, one record per line as  
'name,department,salary':  
Ana,Engineering,7200  
Bo,Sales,5400  
Cy,Engineering,6800  
bad_line_here  
Dia,Sales,5900  
Eva,Support,4700
```

EXPECTED OUTPUT

```
report.txt containing per-department employee counts and  
average salaries, e.g.:  
Engineering: 2 employees, avg salary 7000.00  
Sales: 2 employees, avg salary 5650.00  
Support: 1 employees, avg salary 4700.00  
Plus a console line noting 1 malformed record was skipped.
```

HINTS

- `defaultdict(list)` avoids checking 'if department not in data' before appending
- `Counter(employee_departments)` gives you per-department counts in one line
- Build the report as a list of strings and write it with `writelines()`, or join with `'\n'` and `write()` once

OPTIONAL CHALLENGE

Extend the report to also include the single highest-paid employee overall, using a nested dict lookup rather than re-scanning the raw file.